

Software Design Specification (SDS)

=====

Project Title:

Advanced Tic Tac Toe with SQLite and AI

Overview:

This project implements a console-based Tic Tac Toe game that supports:

- User registration and login via an SQLite database.
- Player-vs-Player and Player-vs-AI modes.
- AI with three difficulty levels: Easy, Medium, and Hard.
- Game result tracking with history saved to a database.

System Requirements:

- C++ Compiler with C++11 or later support.
- SQLite3 library.
- Console environment for input/output.

Main Features:

1. User Authentication:

- Register and login using a username and password.
- Passwords are hashed using a simple custom hash function.

2. Game Modes:

- Player vs Player (each user must log in or register).
- Player vs AI (select difficulty).

3. AI Difficulty:

- Easy: Random moves.
- Medium: Block player or win if possible.
- Hard: Minimax algorithm for optimal strategy.

4. Game History:

- Stores game result (win/loss/draw) along with the username and timestamp.

Design and Architecture:

- Procedural design with global variables and helper functions.
- Central main() function controls user flow and mode selection.
- SQLite handles persistent data storage.

Modules and Functional Units:

1. Database Module:

- setupDatabase(): Creates required tables.
- executeSQL(sql): Executes a given SQL query safely.
- saveGameHistory(result, user): Saves a game result.
- viewHistory(): Displays the last 20 records.

2. User Module:

- registerUser(username): Handles new user registration.
- loginUser(username): Validates existing user credentials.

3. Game Logic:

- displayBoard(): Prints the game board.
- checkWinner(): Checks for a winning player.
- isDraw(): Checks for a draw.
- playGame(): Main game loop for both PVP and PVE.

4. AI Module:

- easyAIMove(): Random selection.
- mediumAIMove(): Basic block/win logic.
- hardAIMove(): Uses Minimax to choose optimal move.
- minimax(): Recursive evaluation for hard AI.

5. Main Flow Control:

- Prompts user for registration/login.
- Offers menu for game mode selection.
- Manages player turns and invokes the appropriate AI or player input.
- Handles results and updates database accordingly.

Database Schema:

1. Users Table

- id: INTEGER PRIMARY KEY AUTOINCREMENT
- username: TEXT UNIQUE

- password: TEXT (hashed)

2. History Table

- id: INTEGER PRIMARY KEY AUTOINCREMENT
- username: TEXT
- result: TEXT
- date: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Assumptions and Limitations:

- The password hash function is simplistic and should not be used for production.
- The database is stored locally as game.db.
- The application is single-threaded and console-based.
- No encryption or network security is implemented.